ETAPA 03 · EXERCÍCIO PROGRESSIVO

Estacionamento do shopping
testes isolados com dummy e fake

3 horas · individual ou em dupla · 7 níveis de dificuldade crescente

Como vai funcionar

- O exercício tem **níveis**. Cada nível traz um objetivo, uma tarefa e um **checkpoint**.
- Só avance quando o checkpoint estiver cumprido (**testes verdes**).
- O professor libera o próximo nível quando a maioria da turma chegar ao checkpoint.
- Terminou antes? Vá para o **Nível 7 — Desafios**.

Em dupla?

- **Piloto** digita; **navegador** lê a regra, revisa e pensa no próximo teste.
- Troquem de papel **a cada nível**.
- Um único repositório Git; um commit por nível concluído.

Individual?

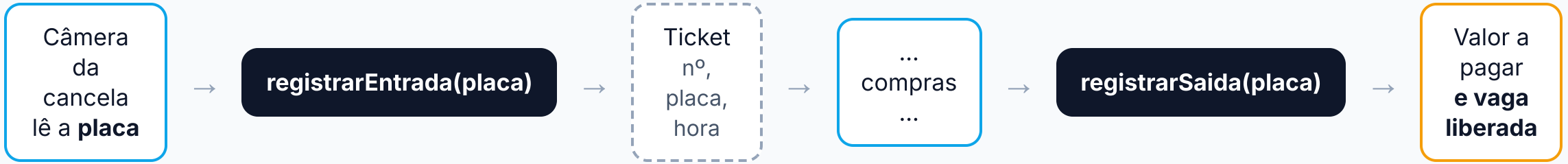
- Faça um commit por nível; ele será sua evidência de progresso.

Mapa da aula

Nível	Tema	Dublê	Tempo
0	Preparação e leitura do código	—	10 min
1	Decompor requisitos e mapear dependências	—	25 min
2	Testes de validação sem tocar em nada	**Dummy**	20 min
3	Entrada de veículos com um banco em memória	**Fake**	30 min
	Intervalo		10 min
4	Tabela de preços com um relógio controlável + chamado	**Fake**	30 min
5	Mensalistas sem depender do ERP	**Fake**	20 min
6	Quem testa o fake? Teste de contrato	**Fake**	20 min
7	Desafios (para quem terminar antes)	ambos	bônus
	Abertura e fechamento		15 min

O cenário: estacionamento do shopping

A administradora do shopping vai trocar o sistema das cancelas. O novo back-end controla **entradas**, **saídas** e a **cobrança**.



RF01 · Registrar entrada

Valida a placa, verifica lotação e emite um ticket.

RF02 · Registrar saída

Calcula o valor pela tabela (ou isenta o mensalista), fecha o ticket e libera a vaga.

Regras de negócio

Código	Regra	Reação
RN01	Placa obrigatória, padrão antigo (`ABC1234`) ou Mercosul (`ABC1D23`). Hífen e minúsculas são aceitos e normalizados.	`IllegalArgumentException`
RN02	Capacidade de pelo menos 1 vaga.	`IllegalArgumentException`
RN03	Uma placa não pode ter dois tickets abertos.	`VeiculoJaEstacionadoException`
RN04	Lotado: ninguém entra — nem mensalistas.	`EstacionamentoLotadoException`
RN05	Saída de placa sem ticket aberto é recusada.	`TicketNaoEncontradoException`
RN06	Até **15 min** (inclusive) é grátis. Conta-se em minutos completos.	—
RN07	Acima da tolerância: **primeira hora R$ 10,00**.	—
RN08	Cada **hora adicional ou fração**: **R$ 5,00**.	—
RN09	Teto de **R$ 50,00**.	—
RN10	Mensalista **em dia na data da saída** não paga; vencido paga como avulso.	—
RN11	Na saída, o ticket é fechado (hora e valor) e a vaga é liberada.	—

A tabela de preços em exemplos

Permanência	Valor
10 min	R$ 0,00 (tolerância)
15 min	R$ 0,00 (tolerância)
16 min	R$ 10,00
1 h	R$ 10,00
2 h	R$ 15,00
10 h	R$ 50,00 (teto)

Valores em centavos

O sistema usa `int` em centavos: R$ 10,00 = `1000`. Em Java, você pode escrever `10_00`.

Por quê? `double` não representa 0,1 exatamente: `0.1 + 0.2` resulta em `0.30000000000000004`.

> "Hora adicional **ou fração**": pense em quais são os valores-limite dessa regra.

O código-base

O código de produção **está pronto**. Seu trabalho é especificá-lo e verificá-lo com testes.




```
estacionamento/
├── pom.xml                (JUnit 5, JaCoCo)
├── README.md              (regras de negócio)
└── src/
    ├── main/java/br/edu/infnet/estacionamento/
    │   ├── EstacionamentoService.java
    │   ├── Ticket.java
    │   ├── Mensalista.java
    │   ├── RepositorioDeTickets.java
    │   ├── CadastroDeMensalistas.java
    │   ├── VeiculoJaEstacionadoException.java
    │   ├── EstacionamentoLotadoException.java
    │   ├── TicketNaoEncontradoException.java
    │   └── infra/   ← produção: NÃO use nos testes
    │       ├── RepositorioDeTicketsEmArquivo.java
    │       └── CadastroDeMensalistasErp.java
    └── test/java/br/edu/infnet/estacionamento/  (vazio)
```

Lendo o serviço: entrada e saída (1/2)

ESTACIONAMENTOSERVICE.JAVA

```java
public Ticket registrarEntrada(String placaInformada) {
    String placa = normalizar(placaInformada);
    if (tickets.buscarAbertoPorPlaca(placa) != null) {
        throw new VeiculoJaEstacionadoException("O veículo " + placa + " já está no estacionamento");
    }
    if (tickets.contarAbertos() >= capacidade) {
        throw new EstacionamentoLotadoException("Estacionamento lotado");
    }
    Ticket ticket = new Ticket(proximoNumero, placa, Instant.now(relogio));
    proximoNumero++;
    tickets.salvar(ticket);
    return ticket;
}
```

Lendo o serviço: entrada e saída (2/2)

```
public Ticket registrarSaida(String placaInformada) {
    String placa = normalizar(placaInformada);
    Ticket ticket = tickets.buscarAbertoPorPlaca(placa);
    if (ticket == null) {
        throw new TicketNaoEncontradoException("Não há ticket aberto para " + placa);
    }
    Instant agora = Instant.now(relogio);
    long minutos = Duration.between(ticket.getEntrada(), agora).toMinutes();

    Mensalista mensalista = mensalistas.buscarPorPlaca(placa);
    boolean isento = mensalista != null && mensalista.emDiaEm(LocalDate.now(relogio));
    int valor = isento ? 0 : calcularValor(minutos);

    ticket.fechar(agora, valor);
    tickets.salvar(ticket);
    return ticket;
}
```

Lendo o serviço: validação e cálculo

```java
/** Aceita "abc-1d23" e devolve "ABC1D23". Formatos aceitos: ABC1234 (antigo) e ABC1D23 (Mercosul). */
private String normalizar(String placa) {
    if (placa == null) {
        throw new IllegalArgumentException("A placa é obrigatória");
    }
    String normalizada = placa.replace("-", "").trim().toUpperCase();
    if (!normalizada.matches("[A-Z]{3}[0-9][A-Z0-9][0-9]{2}")) {
        throw new IllegalArgumentException("Placa inválida: " + placa);
    }
    return normalizada;
}
```

```java
private int calcularValor(long minutos) {
    if (minutos <= TOLERANCIA_EM_MINUTOS) {
        return 0;
    }
    if (minutos <= 60) {
        return PRIMEIRA_HORA_EM_CENTAVOS;
    }
    long horasAdicionais = (minutos - 60) / 60;
    long valor = PRIMEIRA_HORA_EM_CENTAVOS + horasAdicionais * HORA_ADICIONAL_EM_CENTAVOS;
    return (int) Math.min(valor, TETO_EM_CENTAVOS);
}
```

Os dois dublês de hoje

Dummy

Objeto que **só ocupa o lugar** de um parâmetro obrigatório. O código testado **nunca o usa**.

Pode ser `null`, uma classe vazia ou — a forma mais segura — uma classe que **falha se for usada**.

Fake

Implementação que **funciona de verdade**, mas de forma simplificada: um banco em memória, um relógio que o teste adianta.

O teste interage com ele pela interface normal — como faria com o objeto real.

> Existem outros dublês (*stub*, *spy*, *mock*). Hoje o desafio é resolver tudo **só com dummy e fake**. Não use bibliotecas de *mock*.

NÍVEL 0

Preparação

Abrir o projeto, executar o build e ler o código com olhar de quem vai testá-lo.

⏱ 10 min individual ou dupla

Nível 0 · Preparação ⏱ 10 min

Prepare o ambiente e leia o código

1. Abra `estacionamento/` no IntelliJ (*File → Open*).
2. Execute no terminal:

```
mvn clean test
```

3. Leia o `README.md` e o `EstacionamentoService`.
4. Crie o repositório Git e faça o primeiro commit.

Responda (anote no README)

1. Quais classes são apenas **dados**?
2. Por que não dá para usar `CadastroDeMensalistasErp` nos testes?
3. E o `RepositorioDeTicketsEmArquivo`: ele funciona, então qual seria o problema de usá-lo?
4. Por que o relógio é recebido no construtor?

✅ **Checkpoint 0:** `BUILD SUCCESS` (com zero testes) e as 4 respostas anotadas.

NÍVEL 1

Decompor

Antes de escrever qualquer teste: quebrar os requisitos em comportamentos pequenos e decidir que colaboradores cada teste realmente usa.

⏱ 25 min

papel, quadro ou README

Nível 1 · Decompor ⏱ 15 min

1A · Árvore de decomposição e matriz

Quebre **RF01** e **RF02** até chegar a **folhas**: comportamentos que cabem em **um teste**.

> **Critério de folha:** *dada uma situação, ao executar uma ação, ocorre um resultado observável.*

Exemplo de ramo:

RF02 · Registrar saída

- **RF02.2 · Calcular o valor**
  - ▸ 15 min é grátis **RN06**
  - ▸ 16 min custa R$ 10,00 **RN07**
  - ▸ ...

Matriz de rastreabilidade

Regra	Folha	Nome do teste
RN06	15 min é grátis	`registrarSaidaDentroDaTolerancia`
RN03	...	...
...	...	...

Convenção (Etapa 02): `operacao` + cenário. Ex.: `registrarEntradaComEstacionamentoLotado`.

Nível 1 · Decompor ⏱ 10 min

1B · Quem cada teste realmente usa?

Para cada grupo de testes, marque se o colaborador é **usado** (vai precisar de um fake) ou **não usado** (basta um dummy).

Grupo de testes	RepositorioDeTickets	CadastroDeMensalistas	Clock
Validação de placa e capacidade (RN01, RN02)	?	?	?
Entrada: duplicidade e lotação (RN03, RN04)	?	?	?
Saída sem ticket (RN05)	?	?	?
Tabela de preços (RN06 a RN09)	?	?	?
Mensalistas (RN10)	?	?	?

> ✅ **Checkpoint 1:** árvore com **pelo menos 14 folhas**, todas as RN01–RN11 na matriz e a tabela acima completa. Commit: `nivel-1: decomposicao`.

NÍVEL 2

Dummy

Algumas regras são verificadas antes de qualquer colaborador ser tocado.
Nesses testes, os colaboradores só precisam existir.

⏱ 20 min três formas de dummy

Nível 2 · Dummy ⏱ 5 min

Três formas de dummy

Forma	Exemplo	Se o código pas usá-lo...
`null`	`new EstacionamentoService(null, null, null, 10)`	`NullPointerEx` confusa
Classe vazia	métodos que não fazem nada	o teste passa s perceber
Classe que falha	métodos lançam `AssertionError`	**falha com men clara**

```java
class RepositorioDeTicketsDummy implements RepositorioDeTickets {

    @Override
    public void salvar(Ticket ticket) {
        throw new AssertionError(
            "Dummy: o repositório não deveria ser usado neste teste");
    }

    // buscarAbertoPorPlaca e contarAbertos: idem
}
```

> Hoje use a terceira forma. Ela também **documenta**: “este teste não depende do banco”.

Nível 2 · Dummy ⏱ 15 min

Crie os dummies e escreva 5 testes

Dummies

- `RepositorioDeTicketsDummy`
- `CadastroDeMensalistasDummy`
- `RelogioDummy` — estende `java.time.Clock` (3 métodos abstratos: `getZone`, `withZone`, `instant`)

Teste	Regra
`criarComCapacidadeZero`	RN02
`registrarEntradaSemPlaca`	RN01
`registrarEntradaComPlacaForaDoPadrao` (`"AB12345"`)	RN01
`registrarEntradaComPlacaCurta` (`"ABC123"`)	RN01
`registrarSaidaComPlacaForaDoPadrao` (`"1234ABC"`)	RN01

Experimente: passe um `RepositorioDeTicketsDummy` e chame `registrarEntrada("ABC1D23")` (placa válida). O que acontece? Por quê?

✅ **Checkpoint 2:** 5 testes verdes usando apenas dummies. Commit: `nivel-2`.

NÍVEL 3

Fake: repositório

Para testar a entrada de veículos, o banco precisa funcionar: lembrar quem entrou e contar as vagas. Mas sem banco de verdade.

⏱ 30 min banco em memória

Nível 3 · Fake: repositório ⏱ 10 min

Construa o repositório em memória

```
class RepositorioDeTicketsEmMemoria
        implements RepositorioDeTickets {

    private final Map<Integer, Ticket> porNumero =
            new LinkedHashMap<>();

    @Override
    public void salvar(Ticket ticket) {
        // complete
    }

    @Override
    public Ticket buscarAbertoPorPlaca(String placa) {
        // complete
    }

    @Override
    public int contarAbertos() {
        // complete
    }
}
```

Leia o contrato da interface

- `salvar`: grava um ticket novo **ou atualiza** um existente (mesmo número).
- `buscarAbertoPorPlaca`: só tickets **abertos**; `null` se não houver.
- `contarAbertos`: veículos dentro do estacionamento.

> Por que um `Map` por número, e não simplesmente uma `List`? Guarde a pergunta para o Nível 6.

Nível 3 · Fake: repositório ⏱ 20 min

Escreva estes 6 testes

Monte o serviço com o **repositório fake**, o **cadastro dummy** (a entrada nunca consulta mensalistas) e `Clock.fixed(Instant.parse("2026-03-10T10:00:00Z"), ZoneOffset.UTC)`. Use capacidade 2.

Teste	O que verifica	Regra
`registrarEntrada`	ticket nº 1, placa, hora de entrada, aberto, uma vaga a menos	RF01
`registrarEntradaComPlacaComHifenEMinusculas`	`"abc-1d23"` fica gravado como `ABC1D23` (consulte o fake)	RN01
`registrarEntradaDeDoisVeiculos`	números sequenciais; nenhuma vaga livre	RF01
`registrarEntradaDeVeiculoJaEstacionado`	recusada; continua 1 ticket aberto	RN03
`registrarEntradaComEstacionamentoLotado`	3º veículo é recusado	RN04
`registrarSaidaSemTicket`	`TicketNaoEncontradoException`	RN05

✅ **Checkpoint 3:** 6 testes verdes; nenhum arquivo nem banco criado; o cadastro de mensalistas continua dummy. Commit: `nivel-3`.

Intervalo

☕ 10 minutos

Na volta: como testar “2 horas de permanência” em menos de um segundo?

NÍVEL 4

Fake: relógio

Testar a tabela de preços exige que o tempo passe. Esperar não é opção: o teste vai controlar o relógio.

⏱ 30 min relógio controlável

Nível 4 · Fake: relógio ⏱ 17 min

Um relógio que o teste adianta (e um cadastro vazio)

```
class RelogioAjustavel extends Clock {

    private Instant agora;

    RelogioAjustavel(String instanteIso) { /* ... */ }

    void avancar(Duration tempo) { /* ... */ }

    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId z) { return this; }
    @Override public Instant instant() { return agora; }
}
```

A saída **consulta o cadastro de mensalistas**: aqui o dummy não serve mais. Crie `CadastroDeMensalistasEmMemoria` (um `Map` por placa com `cadastrar(Mensalista)`); por enquanto, vazio.

Teste	Permanência → valor
`registrarSaidaDentroDaTolerancia`	15 min → 0
`registrarSaidaLogoAposATolerancia`	16 min → 10_00
`registrarSaidaComUmaHora`	60 min → 10_00
`registrarSaidaComDuasHoras`	2 h → 15_00
`registrarSaidaAcimaDoTeto`	10 h → 50_00
`registrarSaida`	ticket fechado, hora de saída, vaga livre
`registrarEntradaAposSair`	mesma placa entra de novo

Dica: um método auxiliar `valorAposPermanecer(Duration)` evita repetição.

Nível 4 · Fake: relógio ⏱ 13 min

🎫 Chamado do suporte #2093

De: Ouvidoria do shopping · **Prioridade:** alta

"A auditoria comparou os tickets com o caixa: um cliente ficou **1 h 40 min** e pagou **R$ 10,00**. Pela tabela, deveria pagar R$ 15,00. A diferença aparece em muitos tickets do mês."

Sua tarefa

1. Escreva `registrarSaidaComUmaHoraEQuarentaMinutos` reproduzindo o chamado. Ele deve ficar **vermelho**.
2. Escreva os testes dos **valores-limite** da regra "ou fração" (1h01 e 2h01).
3. Encontre o defeito no código de produção, corrija e deixe tudo **verde**.

Por que os 5 primeiros testes não pegaram o defeito? Olhe para as permanências escolhidas.

✅ **Checkpoint 4:** 7 testes + 3 testes do chamado (vermelho → verde), defeito corrigido. Commit: `nivel-4: corrige #2093`.

NÍVEL 5

Fake: cadastro

O ERP só funciona dentro da rede do shopping e leva 3 segundos por consulta. O fake do Nível 4 vai ganhar mensalistas.

⏱ 20 min dados controlados pelo teste

Nível 5 · Fake: cadastro ⏱ 20 min

Escreva estes 6 testes

Use `CadastroDeMensalistasEmMemoria`, relógio em **10/03/2026 10:00 UTC**, permanência de **3 horas** e capacidade 1.

Teste	Situação	Esperado
`registrarSaidaDeMensalistaEmDia`	pago até daqui a 20 dias	R$ 0,00
`registrarSaidaDeMensalistaNoUltimoDiaPago`	pago até hoje	?
`registrarSaidaDeMensalistaVencido`	pago até ontem	?
`registrarSaidaDeOutraPlacaDoMensalista`	o cadastro tem outra placa	?
`registrarEntradaDeMensalistaComEstacionamentoLotado`	a única vaga está ocupada	?
`registrarSaidaDeMensalistaQueVenceuDuranteAPermanencia`	entrou 31/03 às 22h, pago até 31/03, saiu 01/04 à 1h	?

O último teste é de **caracterização**: registre o que o código faz **hoje**. Você concorda com esse comportamento? O que o cliente diria?

✅ **Checkpoint 5:** 6 testes verdes, nenhum acesso ao ERP, resposta da discussão anotada. Commit: `nivel-5`.

NÍVEL 6

Contrato do fake

Todos os testes confiam no repositório em memória. E se ele estiver errado?
Um fake também é código — e também precisa de testes.

⏱ 20 min teste de contrato

Nível 6 · Contrato do fake ⏱ 15 min

Quem testa o fake?

Escreva uma classe **abstrata** com os testes que **qualquer** `RepositorioDeTickets` deve passar:

```
abstract class ContratoDoRepositorioDeTickets {

    private RepositorioDeTickets repositorio;

    protected abstract RepositorioDeTickets criarRepositorio();

    @BeforeEach
    void criar() {
        repositorio = criarRepositorio();
    }

    // testes do contrato aqui
}

class RepositorioDeTicketsEmMemoriaTest
        extends ContratoDoRepositorioDeTickets {
    @Override
    protected RepositorioDeTickets criarRepositorio() {
        return new RepositorioDeTicketsEmMemoria();
    }
}
```

Teste do contrato
`buscarAbertoDePlacaSemTicket`
`salvarEBuscarAberto`
`buscarAbertoIgnoraTicketFechado`
`contarAbertosIgnoraFechados`
`salvarNovamenteAtualiza` — salve o ticket 1 aberto e depois **outro objeto** com o mesmo número, fechado

> Tickets fechados podem ser criados com o construtor completo: `new Ticket(1, placa, entrada, saida, valor)`.

Nível 6 · Contrato do fake ⏱ 5 min

Experimento: um fake ingênuo

Troque temporariamente o `Map` do seu fake por uma `List` em que `salvar` apenas faz `add`. Rode **toda** a suíte.

1. Quais testes falham?
2. Por que os testes do serviço (Níveis 3 a 5) continuam verdes?
3. O que aconteceria em produção, com o repositório de arquivo, se o serviço confiasse nesse comportamento?

A ideia central

Um fake só é útil se se comportar **como o real**. O teste de contrato é a garantia disso — e, no Desafio A, o mesmo contrato roda contra a implementação de produção.

✅ **Checkpoint 6:** contrato com 5 testes verdes para o fake, experimento respondido e fake restaurado. Commit: `nivel-6`.

NÍVEL 7

Desafios

Para quem chegou até aqui antes do tempo. Escolha um ou mais.

bônus em qualquer ordem

Nível 7 · Desafios ⏱ bônus

Escolha seu desafio

A · Contrato contra o real

Crie `RepositorioDeTicketsEmArquivoTest` estendendo o contrato e usando `@TempDir` para o arquivo.

Se fake e real passam no mesmo contrato, os testes que usam o fake valem também para o real.

B · Auxiliar de teste

Crie `CenarioDeEstacionamento`, um *test data builder* com fakes prontos:

```
umEstacionamento().comMensalistaEmDia("ABC1D23").comVeiculoQueEntrouHa("ABC1D23", Duration.ofHours(5))
```

C · Decidir a RN12

Com base no teste de caracterização do Nível 5, escreva a **RN12**: vale a mensalidade da data de **entrada** ou da **saída**?

Implemente a decisão e transforme o teste de caracterização em teste da regra.

Fechamento: discussão

1. Em quais testes um **dummy** bastou? O que esses testes têm em comum?
2. Se o banco de dados mudar de arquivo para PostgreSQL, quantos testes do serviço mudam?
3. Por que o defeito do chamado #2093 passou pelos primeiros testes da tabela de preços?
4. Qual o risco de um fake mal feito? Como o teste de contrato reduz esse risco?

Entrega

- Repositório Git com **um commit por nível**.
- `mvn clean test` verde.
- README com respostas do Nível 0, árvore + matriz + tabela do Nível 1 e respostas dos Níveis 5 e 6.
- Em dupla: os dois nomes no README.